

Lab 7-1

Message Queue

TA: Kuan-Tsen Liu

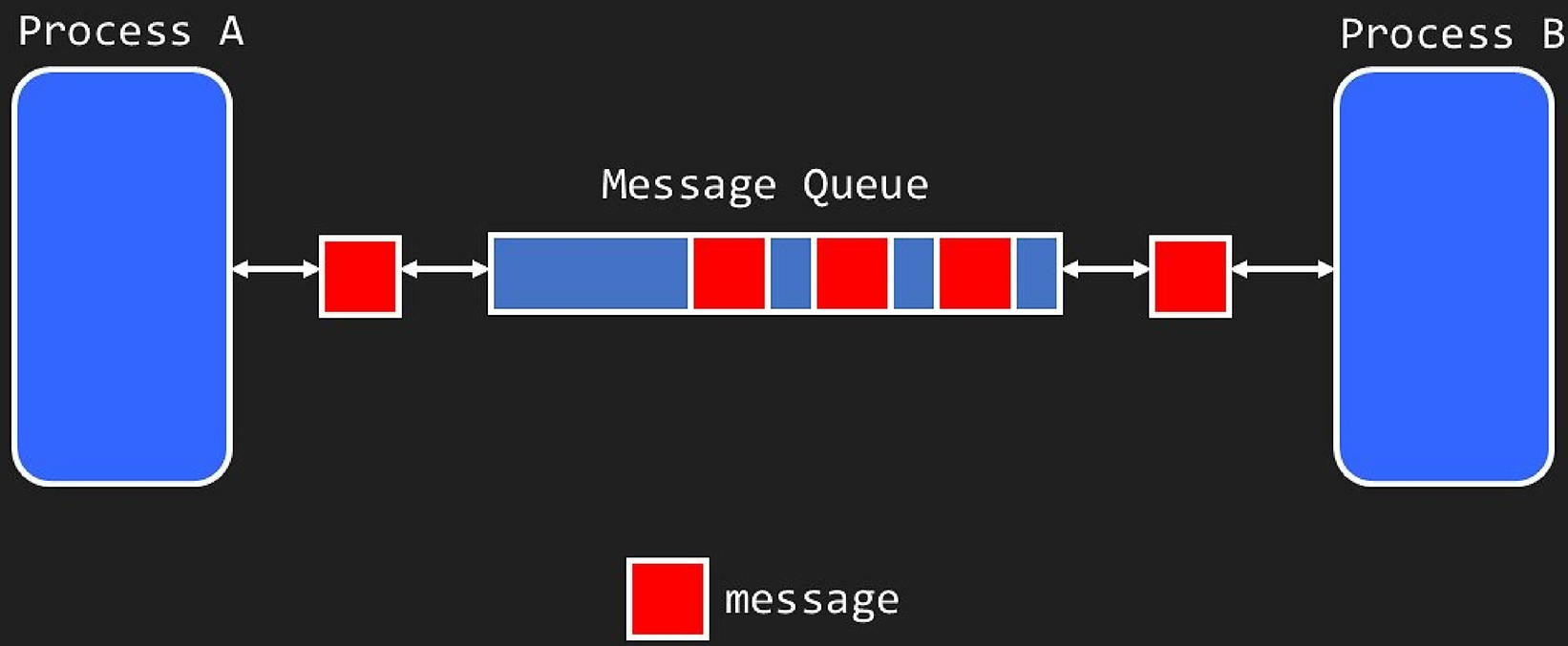
Prof.: Hsung-Pin Chang

$ Outline

- Introduction
- API
- Example
- Exercises
- References

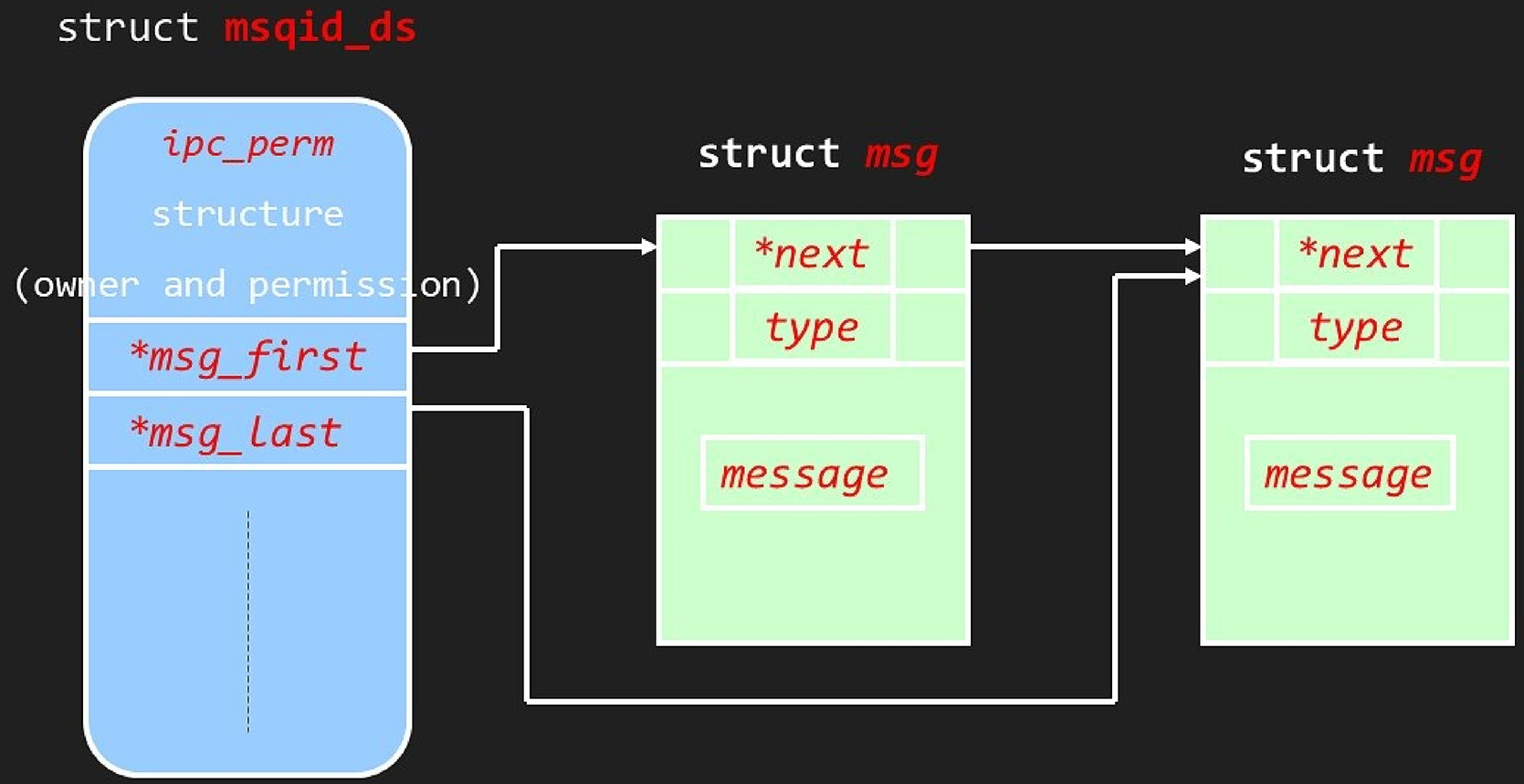
$ Introduction (1/2)

- Provide a reasonably easy and efficient way of passing data between two unrelated processes



$ Introduction (2/2)

- Message Queue Structure in Linux



$ Outline

- Introduction
- API
- Example
- Exercises
- References

$ API (1/9)

- **Main functions in the POSIX message queue API**
  - **mq_open()**: creates a new message queue or opens an existing queue
  - **mq_close()**: closes a message queue
  - **mq_unlink()**: deletes a message queue
  - **mq_send()**: writes a message to a queue
  - **mq_receive()**: reads a message from a queue.

$ API-Opening a Message Queue (2/9)

```
#include <fcntl.h>              /* Defines O_* constants */
#include <sys/stat.h>           /* Defines mode constants */
#include <mqueue.h>

mqd_t mq_open(const char *name, int oflag, ...
              /* mode_t mode, struct mq_attr *attr */);
```

Returns a message queue descriptor on success, or *(mqd_t)* −1 on error

- **oflag:** as shown in the next slide
- **mode:** as shown in the following next slide
- **attr:** attributes for the new message queue
  - Shown in the following next slide
- **mqd_t:** returns a message queue descriptor, a value of type mqd_t
  - mqd_t is an int in Linux

$ API (3/9)

- Bit values for the mq_open() oflag argument

Flag	Description
O_CREAT	Create queue if it doesn't already exist
O_EXCL	With O_CREAT, create queue exclusively
O_RDONLY	Open for reading only
O_WRONLY	Open for writing only
O_RDWR	Open for reading and writing
O_NONBLOCK	Open in nonblocking mode

$ API (4/9)

- mode argument :
  - Library sys/stat.h
  - Constants for File Permission Bits

Constant	Octal value	Permission bit
S_IRUSR	0400	User-read
S_IWUSR	0200	User-write
S_IXUSR	0100	User-execute
S_IRGRP	040	Group-read
S_IWGRP	020	Group-write
S_IXGRP	010	Group-execute
S_IROTH	04	Other-read
S_IWOTH	02	Other-write
S_IXOTH	01	Other-execute

$ API (5/9)

- Message Queue Attributes

```
struct mq_attr {
    long mq_flags;       /* Message queue description flags: 0 or
                            O_NONBLOCK [mq_getattr(), mq_setattr()] */
    long mq_maxmsg;      /* Maximum number of messages on queue
                            [mq_open(), mq_getattr()] */
    long mq_msgsize;     /* Maximum message size (in bytes)
                            [mq_open(), mq_getattr()] */
    long mq_curmsgs;     /* Number of messages currently in queue
                            [mq_getattr()] */
};
```

- `mq_open()` only needs to specify two of the fields
  - `mq_maxmsg` and `mq_msgsize`
  - As shown in the comments

$ API-Closing a Message Queue (6/9)

mq_close(): closes a message queue

```
#include <mqueue.h>

int mq_close(mqd_t mqdes);
```

Returns 0 on success, or −1 on error

- **mqd_t:** returns a message queue descriptor
  - mqd_t is an integer in Linux
- **mq_close() doesn't delete queue.**

$ API-Removing a Message Queue(7/9)

mq_unlink(): deletes a message queue

```
#include <mqueue.h>

int mq_unlink(const char *name);
```

Returns 0 on success, or −1 on error

- **Removes the queue's name**
  - If O_CREAT flag is not set , thread can't use the queue by name
- **Destroys the queue**
  - Accurately if the reference count falls to zero, or
  - Postponed until all references to the message queue have been closed

$ API-Sending Messages (8/9)

mq_send(): writes a message to a queue

```
#include <mqueue.h>

int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len,
            unsigned int msg_prio);
```

Returns 0 on success, or −1 on error

- **mqdes:** descriptor referred to the message queue
- **msg_ptr:** pointer points to the message
- **msg_len:** the length of message pointed to by msg_ptr
- **msg_prio:** NULL (Here, we ignore the message priority)

$ API-Receiving Messages (9/9)

mq_receive(): reads a message from a queue.

```
#include <mqueue.h>

ssize_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len,
                   unsigned int *msg_prio);
```

Returns number of bytes in received message on success, or −1 on error

- **msg_ptr**: buffer pointed to by msg_ptr
- **msg_len**: the number of bytes of space available in the buffer
  - hence, the size of buffer must be greater than or equal to the mq_msgsize of the queue
- **msg_prio**: NULL (Here, we ignore the message priority)

$ Outline

- Introduction
- API
- Example
- Exercises
- References

$ Example-Directory Structure(1/4)

- lab7 (資料夾)
  - example1 (資料夾)
    - sender.c
    - receiver.c

$ Example (2/4)

- lab7/example1/sender.c

String ends with '\0'

```c
#include<fcntl.h>      // defines O_* constants for mq_open()
#include<mqueue.h>
#include<stdio.h>
#include<string.h>

#define NAME "/myMsgQueue"
#define MAX_MSGSIZE 1000

int main() {
    mqd_t mqd = mq_open(NAME, O_WRONLY | O_CREAT, 0644, NULL);
    if (mqd == -1) {
        perror("Failed to open or create the message queue. \n");
        return 1;
    }

    printf("Send to receiver: \n");
    while (1) {
        /* initialize buffer array */
        char buffer[MAX_MSGSIZE];
        for (int i = 0; i < sizeof(buffer); i++)
            buffer[i] = '\0';

        /* let user type message */
        printf("> ");
        fgets(buffer, MAX_MSGSIZE, stdin);

        /* send the message: 0 -> success, -1 -> fail */
        int send = mq_send(mqd, buffer, strlen(buffer) + 1, 0);
        if (send == -1) {
            perror("Failed to send a message. \n");
            return 1;
        }
    }

    return 0;
}
```

$ Example (3/4)

- lab7/example1/receiver.c

```c
#include<fcntl.h>
#include<mqueue.h>
#include<stdio.h>

#define NAME "/myMsgQueue"
#define MAX_MSGSIZE 1000

int main() {
    mqd_t mqd = mq_open(NAME, O_RDONLY | O_CREAT, 0644, NULL);
    if (mqd == -1) {
        perror("Failed to open or create the message queue. \n");
        return 1;
    }

    while (1) {
        char buffer[MAX_MSGSIZE];
        ssize_t msg_size =  mq_receive(mqd, buffer, 8192, NULL);
        if (msg_size == -1) {
            perror("Failed to read the message. \n");
            return 1;
        }
        printf("received: %s", buffer);
    }

    /* close the message queue: 0 -> success, -1 -> fail */
    int close = mq_close(mqd);
    if (close == -1) {
        perror("Failed to close the message queue. \n");
        return 1;
    }

    /* unlink the message queue: 0 -> success, -1 -> fail */
    int unlink = mq_unlink(NAME);
    if (unlink == -1) {
        perror("Failed to unlink the message queue. \n");
        return 1;
    }

    return 0;
}
```

$ Example (4/4)

- Result



$ Outline

- Introduction
- API
- Example
- Exercises
- References

$ Exercise-Directory Structure(1/7)

- lab7 (資料夾)
  - example1 (資料夾)
    - sender.c
    - receiver.c
  - exercise1 (資料夾)
    - sender.c
    - receiver.c

$ Exercise-Lab1 (50 pts.)

Write two programs:

- sender.c
  - create a message queue
  - user can iteratively type strings, until typing “exit”
  - send message to message queue
- receiver.c
  - iteratively receive message from message queue
  - show the message on the screen
  - close & unlink the message queue

$ Exercise-Lab1

- The results are as follows :



$ Exercise-Lab1

- You might use these functions:
  - fgets(char *s, int n, FILE *stream)
    - Get string from *stream to *s with length n. Use stdin in *stream filed to get input from keyboard.
    - Ex: fgets(buf, BUFSIZ, stdin)
  - strcmp(string str1, string str2)
    - Compare str1 and str2. Return 0 if equal.
    - Ex: strcmp(message, “hello”)

$ Outline

- Introduction
- API
- Example
- Exercises
- References

$ References

The Linux Tutorial

https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_shared_memory.htm

https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_message_queues.htm

The Linux Programmer's Guide

http://tldp.org/LDP/lpg/



Thanks for your listening

Made by Jerry Liu in 2026/10